

National Chi-Nan University
Department of Information Management

Operating Systems

2016/11/18

Note: 3 pages in total.

I. Selection. (Multiple choices are possible) (10%)

1. Which of the following components of program state are not shared among threads in a multithreaded process? (A) global variables (B) CPU register values (C) stack memory (D) heap memory (E) program code
BCE
2. Consider a multithreaded program running on a multicore system. Threads can run in parallel on the system for which multithreaded models? (A) many-to-one model (B) one-to-one model (C) many-to-many model (D) one-to-many model (E) two-level model
BCE
3. Consider a multithreaded program running on a multicore system. If a thread makes a blocking system call, the entire process will block for (A) many-to-one model (B) one-to-one model (C) many-to-many model (D) one-to-many model (E) two-level model
A
4. A multithreaded program can achieve better performance when running on a multiprocessor system than on a single-processor system for (A) many-to-one model (B) one-to-one model (C) many-to-many model (D) one-to-many model (E) two-level model
BCE
5. For nonpreemptive scheduling, CPU-scheduling decisions may take place when a process (A) enters the system (B) switches from running to waiting state (C) switches from running to ready state (D) switches from waiting to ready state (E) terminates.
BCE

II. Fill in the Blank. (Answer terminologies in English) (16%)

1. An initial program that initializes all aspects of the system, loads OS kernel, start executing the OS is bootstrap.
2. What virtualization software do you use to build your virtual machines?
VMware Parallels
3. An embedded system is a combination of computer hardware and software, and perhaps additional mechanical or other parts, designed to perform specific tasks.
4. System call allow user programs to request services of OS.

5. A command interpreter that gets and executes the next user-specified command is shell.

6. A multiprocessor system in which each processor performs all tasks within OS is thread symmetric multiprocessing.

7. DMA I/O: Device controller transfers an entire block of data directly to or from its own buffer storage to main memory, without CPU intervention.

8. In a multiprogramming system, OS keeps several jobs in memory simultaneously. When one job being executed needs to wait, the OS switches to and executes another job.

III. Questions. (Answer in Chinese or English) (74%)

1. (12%) What is the key difference between a trap and an interrupt? (4%)

What is dual-mode operation? (2%) Using trap and timer interrupt as examples to discuss why modern operating systems take advantage of hardware-supported dual-mode operation. (6%)

2. (8%) Discuss the difference between microkernel and monolithic kernel structure.

3. (16%) What is the modular kernel approach in operating-system design? Compare its advantages over monolithic kernel, microkernel, and layered approach. (4%)(12%)

4. (12%) Please take the producer-consumer problem as an example to discuss the strengths and weaknesses of using two different models of interprocess communication. Draw the diagram for each model. (8%)(4%)

5. (16%) A system distinguishes two types of processes: real-time process and normal process. Basically, the system uses preemptive priority-based scheduling algorithm. Real-time processes are assigned with the highest priority and scheduled in FCFS order. Normal processes are assigned with the lowest priority and scheduled in Round-Robin order with time quantum 2. The table below shows five processes that need to be scheduled. The length of the CPU-burst time is given in milliseconds.

(a) Please draw the Gantt chart. (5%)

(b) Please show the waiting time for each process and show average waiting time. (10%)(1%)

Process	Arrival Time	CPU Burst	Type
P1	1	6	Normal
P2	9	2	Real-time
P3	0	3	Normal
P4	7	3	Real-time
P5	4	1	Real-time

6. (10%) What would be the output for the following C program using UNIX `fork()` and Pthreads API?


```
#include <pthread.h>
#include <stdio.h>
```

```
int value = 0;
```

```
void runner()
```

```
{
    printf("(1) %d", value);
    value++;
}
```

```
void *start(void *param)
```

```
{
    runner();
    value++;
    pthread_exit(0);
}
```

```
int main(int argc, char *argv[])
```

```
{
    int pid;
    pthread_t tid;
    pthread_attr_t attr;
```

```
    value++;
```

```
    pid = fork();
```

```
    value++;
```

```
    if (pid == 0) { /* child process */
```

```
        value++;
```

```
        printf("(2) %d", value);
```

```
        pthread_attr_init(&attr);
```

```
        value++;
```

```
        pthread_create(&tid, &attr, start, NULL);
```

```
        value++;
```

```
        pthread_join(tid, NULL);
```

```
        runner();
```

```
    } else if (pid > 0) { /* parent process */
```

```
        value++;
```

```
        wait(NULL);
```

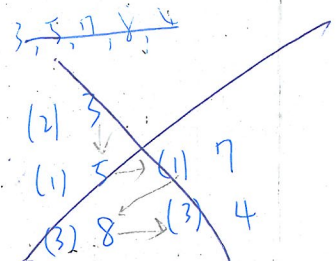
```
        value++;
```

```
    printf("(3) %d", value);
```

```
}
```

a. (2) 3 b. (1) 5
c. (1) 6 d. (3) 6
e. (3) 4.

(1) 0 (2) 1 (3) 4 (3) 1



(2) 3 (1) 5 (1) 7 (3) 8 (3) 4

National Chi-Nan University
Department of Information Management

Operating Systems

2017/11/23

Note: 3 pages in total.

I. Fill in the Blank. (Answer terminologies in English only) (20%)

1. A _____ system can suffer a failure of any single component and still continue operation.
2. A clustered system composes of two or more individual systems or nodes joined together. It is usually used to provide high availability or high-performance computing environments.
3. What virtualization software do you use to build your virtual machines? Parallels Desktop
4. In a _____ system, all processors are peers and no boss-worker relationship exists between processors. Therefore, each processor performs all tasks within OS.
5. An embedded system is a combination of computer hardware and software, and perhaps additional mechanical or other parts, designed to perform specific tasks.
6. In a _____ system, OS keeps several jobs in memory simultaneously. When one job being executed needs to wait, the OS switches to and executes another job.
7. In a monolithic system, OS contains a large amount of functionality to be combined into one level. OS is written as a collection of procedures, each of which can call any of the other ones whenever it needs to.
8. An initial program that initializes all aspects of the system, loads OS kernel, start executing the OS is bootstrap.
9. When arranging the CPU to run another process, the system must save the state of the old process and load the saved state for the new process via a _____.
10. Consider a multithreaded program running on a multicore system. If a thread makes a blocking system call, the entire process will block for the many-to-one model.

II. Questions. (Answer in Chinese or English) (80%)

1. (13%) What is dual-mode operation? Using trap, system call, and timer interrupt as examples to discuss why modern operating systems take advantage of hardware-supported dual-mode operation. (4%)(9%)
2. (16%) Describe the microkernel and the modular kernel approach in operating-system design. Compare their strengths and weaknesses in terms of extensibility. (8%)(8%)

3. (12%) Please take the producer-consumer problem as an example to discuss the strengths and weaknesses of using two different models of interprocess communication to solve the producer-consumer problem. Draw the diagram for each model. (8%)(4%)
4. (8%) What is the one-to-one multithreading model? Discuss its strengths and weaknesses. (2%)(6%)
5. (10%) What would be the output for the following C program using UNIX *fork()* and Pthreads API?

```

#include <pthread.h>
#include <stdio.h>

int value = 1;

void runner()
{
    value++;
    printf("(a) %d ", value);
}

void *start(void *param)
{
    runner();
    value++;
    pthread_exit(0);
}

int main(int argc, char *argv[])
{
    int pid;
    pthread_t tid;
    pthread_attr_t attr;

    value++;
    pid = fork();
    value++;
    if (pid == 0) { /* child process */
        value++;
        printf("(b) %d ", value);
        pthread_attr_init(&attr);
        value++;
        pthread_create(&tid, &attr, start, NULL);
        pthread_join(tid, NULL);
        value++;
    }
    else if (pid > 0) { /* parent process */
        value++;
        wait(NULL);
        runner();
    }
    value++;
    printf("(c) %d ", value);
}

```

Handwritten annotations:

- Next to `#include <pthread.h>`: (3) (4)
- Next to `int value = 1;`: (a) (b)
- Next to `void runner()`: (b) (b)
- Next to `printf("(a) %d ", value);`: (4)
- Next to `void *start(void *param)`: (b) 4 (a) 6 (b) 10 (a) 4 (c) 5
- Next to `pthread_exit(0);`: (b) 4 (a) 6
- Next to `if (pid == 0) {`: (4)
- Next to `printf("(b) %d ", value);`: (4)
- Next to `printf("(c) %d ", value);`: (5)
- Handwritten note: *forks 4, 5*

6. (4%) Including the initial process, how many processes are created by the following program? Explain your answer.

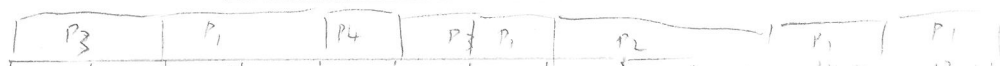
```
#include <stdio.h>
#include <unistd.h>
void main() {
    int i;
    for (i=0; i < 2; i++)
        fork();
    return;
}
```

7. (9%) A system distinguishes two classes of processes: real-time process and normal process. Basically, the system uses **preemptive priority-based** scheduling algorithm. Real-time processes are assigned with the highest priority and scheduled in FCFS order. Normal processes are assigned with the lowest priority and scheduled in Round-Robin order with time quantum 2. The table below shows four processes that need to be scheduled. The length of the CPU-burst time is given in milliseconds.

(a) Please draw the Gantt chart. (4%)

(b) Please show the waiting time for each process and the average waiting time. (5%)

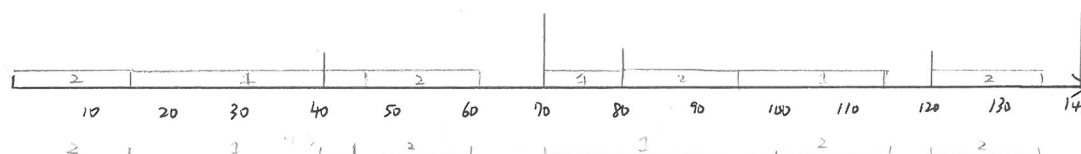
Process	Arrival Time	CPU Burst	Class
P1	1	7 - 2 - 1 4	Normal
P2 ✓	7	3 ✓	Real-time
P3 ✓	0	3 - 2 - 1	Normal
P4 ✓	4	1 - 1	Real-time



8. (8%) Consider two processes, P1 and P2, where $p_1=70$, $t_1=30$, $p_2=40$, and $t_2=15$. Using Gantt charts (till time 140) to illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling and rate-monotonic scheduling. (Note: p: period, t: processing time.) (4%)(4%)

$$P_1 = 70 \quad t_1 = 30$$

$$P_2 = 40 \quad t_2 = 15$$



National Chi-Nan University
Department of Information Management

Operating Systems

2017/01/13

Note: 4 pages in total.

I. Selection. (Multiple choices are possible) (12%)

- C 1. Which synchronization mechanism is most suitable to be used on a multiprocessor system if the lock is held by a currently running thread? (a) blocking semaphore (b) disabling/enabling interrupt (c) spinlock (d) condition variables
- D 2. Which synchronization mechanism is most suitable to be used inside the kernel on a single processor system? (a) blocking semaphore (b) monitor (c) spinlock (d) disabling/enabling interrupt
- A 3. Which synchronization mechanism is most suitable to be used on a multiprocessor system if the lock is held by a thread not currently running? (a) blocking semaphore (b) disabling/enabling interrupt (c) spinlock (d) condition variables
- A, B, E 4. A solution to the critical-section problem must satisfy (a) mutual exclusion (b) progress (c) no preemption (d) circular wait (e) bounded waiting
- C, D, E 5. The necessary conditions of a deadlock are (a) bounded waiting (b) progress (c) no preemption (d) circular wait (e) mutual exclusion
- A, C, ~~X~~ 6. External fragmentation would occur in (a) continuous memory allocation (b) paging in (c) segmentation (d) single-partition scheme (e) swapping in

II. Fill in the Blank. (Answer terminologies in English) (14%)

1. Priority Inversion is a scheduling problem when lower-priority process holds a lock needed by higher-priority process. It leads to the situation in which a low-priority task executes while a higher priority task waits on it due to resource contentions.
2. A situation where several processes access and manipulate the same data concurrently, and the outcome of the execution depends on the particular order in which the access takes place, is called a Race Condition.
3. Virtual Memory is a technique that allows the execution of processes that may not be completely in memory. It separates user logical memory from physical memory.
4. Access to a page that is not in memory causes a Hardware trap.
5. Processor Affinity means the scheduler in SMP systems attempts to keep a process running on the same processor, to take advantage of its data being in that processor's cache memory.

Assume:
 • Associative lookup: ϵ time unit
 • Memory cycle time is m time unit
 Hit ratio = α

$$EAT = \frac{(m + \epsilon)\alpha}{TLB\ hit} + \frac{(2m + \epsilon)(1 - \alpha)}{TLB\ miss}$$

Translation look-aside buffer

$$(100)0.8 + (200)(0.2) = 120\ ns$$

6. In a paging system with three-level page tables, suppose TLB hit ratio is 80% and it takes 100 ns to access memory. The effective memory access time is 120 ns.

7. Consider a computer system with a 32-bit virtual address and 4-KB page size. The system supports up to 512-MB of physical memory. Assume each page table entry is 4 bytes. If one-level paging is used, the size of the page table needed is $2^{32}/2^{12} = 2^{20}$ F4MB. If inverted page table is used, the size of the table needed is 384KB.

$$2^{32} / 2^{12} = 2^{20} = 1\ MB$$

$$2^{20} \times 4\ bytes = 4\ MB$$

$$2^{32} / 2^{12} = 2^{20}$$

$$2^{32} / 2^{13} = 2^{19}$$

$$2^{19} \times 4\ bytes = 384\ KB$$

III. Questions. (Answer in Chinese or English) (74%)

Chp 6

1. (12%) Consider two processes, P1 and P2, where $p1=80$, $t1=40$, $p2=50$, and $t2=15$. Using Gantt charts (till time 150) to illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling and rate-monotonic scheduling. (Note: p : period, t : processing time.) (6%)(6%)

Chp 9

2. (9%) Consider the following page reference string:

1, 2, 3, 4, 4, 1, 5, 1, 4, 2, 3, 4, 2, 5

How many page faults would occur for each of the following page replacement algorithms with four page frames? All page frames are initially empty.

- (a) LRU (3%)
 (b) FIFO (3%)
 (c) optimal page replacement algorithm (3%)

3. (6%) Suppose a solution to the readers-writers problem is described below. Determine if it is correct and explain your answer. In the following codes, the *mutex* semaphore is used to ensure mutual exclusion when *readcount* is updated and the *wrt* semaphore is used to ensure mutual exclusion for the writers and used by the first or last reader that enters or exits the critical section. The *mutex* and *wrt* semaphores are initialized to 1 and *readcount* is initialized to 0.

The structure of a writer process:	The structure of a reader process:
<pre>wait(wrt); ... writing is performed ... signal(wrt);</pre>	<pre>wait(mutex); readcount++; signal(mutex); if (readcount == 1) wait(wrt); ... reading is performed ... wait(mutex); readcount--; signal(mutex); if (readcount == 0) signal(wrt);</pre>

X (8%) Draw the diagram for the address translation hardware required in a paging system with TLB. This diagram should depict how a logical address is translated into a physical address.

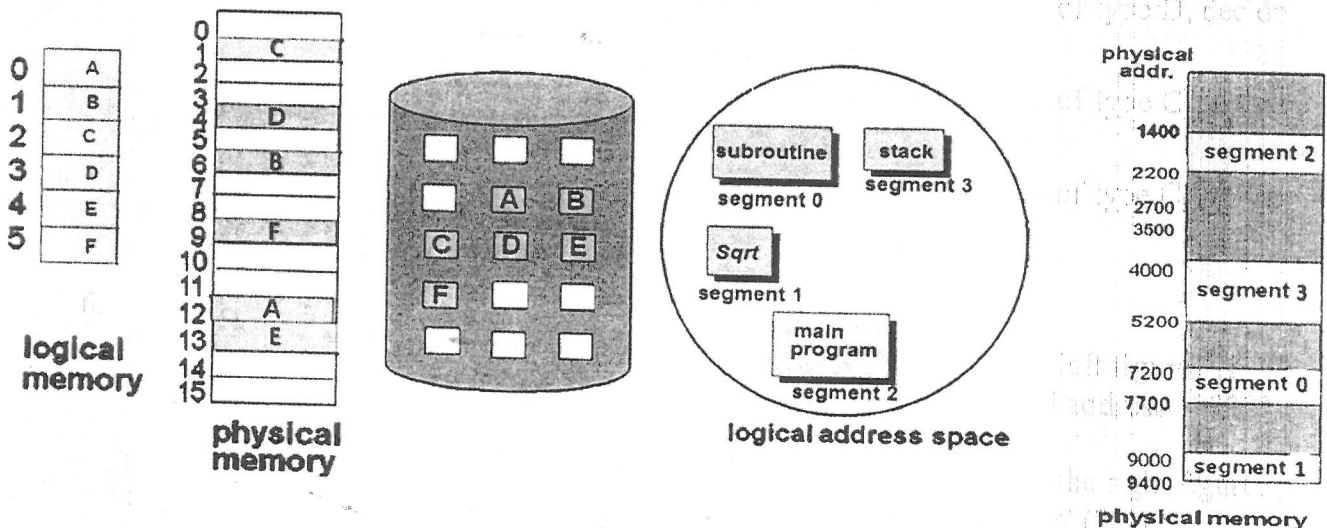
5. (12%) A system has four processes and four resource types A, B, C, D. Resource type A has 5 instances, resource type B has 2 instances, resource type C has 5 instances, and resource type D has 6 instances. The current allocation and maximum needs are as follows:

Process	Current Allocation				Maximum Demand				
	A	B	C	D	A	B	C	D	
P1	1	1	1	1	1	1	2	2	0 0 1 1
P2	2	0	1	1	2	2	2	1	0 2 1 0
P3	1	1	0	1	2	1	3	1	1 0 3 0
P4	1	0	2	1	1	1	2	1	0 1 0 0
Total: 5 2 4 4									

Answer the following questions and explain your answers.

- Determine whether this system is in a safe state. (3%)
- Suppose now that process P1 requests two additional instances of type D, decide whether this request can be immediately granted. (3%)
- Suppose now that process P2 requests one additional instances of type C, decide whether this request can be immediately granted. (3%)
- Suppose now that process P3 requests one additional instances of type C, decide whether this request can be immediately granted. (3%)

- X** (12%)
- What would the page table look like for the process shown in the left figure? If each page is 4 Kbytes, what is the physical address for the logical address 10000? (3%)(3%)
 - What would the segment table look like for the process shown in the right figure? (4%) What is the physical address for the logical address <2,110>? (2%)



7. (6%) Deadlock can occur in the dining-philosophers problem if the philosophers obtain the chopsticks one at a time using semaphore operations. Assume there are five philosophers and five chopsticks. If we allow at most four philosophers to be sitting simultaneously at the table, can it be deadlock-free? Explain your answer and draw the resource allocation-graph. (2%) (4%)

8. (9%)

- (a) Given two processes, the structure of each process is as follows. Explain why it is an incorrect solution to the critical-section problem. (4%)

shared data: int turn; // turn=0 or 1; initially

```
do {
    while (turn != i);
    critical section
    turn = j;
    remainder section
} while (1);
```

process_i

- (b) The following is an incorrect solution of using TestAndSet() instruction to implement semaphore operations wait(s) and signal(s). Please give a scenario showing why it is incorrect. (5%)

```
void wait (s)
{
    while TestAndSet(lock)
        ; // do no-op;
    while (s <= 0)
        ; // do no-op;
    s = s - 1;
    lock = false;
}

void signal (s) {
    while TestAndSet(lock)
        ; // do no-op
    s = s + 1;
    lock = false;
}
```

TRUE